



BFS (Breadth First Search) Graph Traversal



BFS Kya Hota Hai?

Socho agar ek paani ke taalaab mein tumne ek patthar phenka. Kya hota hai? Pehle ek choti si gol lehar (ripple) banti hai, fir uske bahar ek aur badi lehar, aur aise karke wo aage badhti hai. **BFS bilkul waisa hi hai!**

Graph mein hum ek jagah se start karte hain (jaise jahan patthar gira), aur pehle apne sabse paas wale doston (nodes) ko visit karte hain. Phir un doston ke doston ko, aur aise karke aage badhte hain. Isko hum **Level Order Traversal** bhi bolte hain.



Real-Life Examples (Tumhari Life Se!)

Man lo tumhe bhokh lagi hai aur tumne **Zomato** se order kiya. Ya phir **Blinkit** se groceries mangwaye. App ko sabse jaldi pohochne wala rasta (shortest path) dhundhna hai. BFS aise hi kaam karta hai! Woh pehle sabse nazdeek ke routes check karta hai, phir thode aur door ke, taaki tumhara Khana ya Rapido ride sabse jaldi pohnce.

Tumhare decorative lighting business mein jab nayi brass lights ka shipment aata hai, toh usko store karte waqt pehle front row (Level 1) fill karte ho, phir uske peeche wali row (Level 2). Yeh level-by-level arrangement bhi ek tarah ka BFS hi hai!



BFS Kaam Kaise Karta Hai? (Logic)

BFS ko samajhne ke liye humein 2 cheezon ki zarurat hoti hai:

1. **Queue (Line):** Jaise school ki line ya Cult.fit gym mein entry ki line. Jo pehle aayega, woh pehle jayega (FIFO - First In First Out). Hum Python mein `collections.deque` use karte hain.
2. **Visited Array (Tick Mark):** Yeh yaad rakhne ke liye ki kis-kis node se hum mil chuke hain, taaki wapas se ussi se na mile (Infinite loop se bachne ke liye).



Step-by-Step Traversal:

- **Step 1:** Start node ko Queue mein dalo aur usko 'Visited' (True) mark kar do.
- **Step 2:** Queue ke aage (front) se ek node nikalo (popleft) aur usko print karo.
- **Step 3:** Us nikale hue node ke saare padosiyon (neighbors) ko Matrix mein dekho.
- **Step 4:** Agar padosi 'Visited' nahi hai (False), toh usko 'Visited' (True) mark karo aur Queue ke peeche line mein laga do (append).
- **Step 5:** Step 2 se lekar 4 tak repeat karo jab tak Queue पूरी khali na ho jaye!



Levels in BFS (Generation by Generation):

Level 1: [0] (Dada-Dadi)

Level 2: [1, 3] (Unke Bachche)

Level 3: [4, 5] (Bachchon ke Bachche)

Level 4: [6] (Aur aage ki generation)

Bas isi tarah Level by Level aage badhna hi BFS kehlata hai!



Python Code for BFS

Data Analyst ya GenAI Engineer ke interviews mein, ya phir UrbanPro pe apne students ko sikhte waqt, yeh code ekdum perfect kaam aayega! Yahan hum Adjacency Matrix ka use kar rahe hain.

```

from collections import deque

class Graph:
    def __init__(self, size):
        self.size = size
        # 2D Matrix of zeroes (NxN matrix)
        self.matrix = [[0] * size for _ in range(size)]

    def add_edge(self, source, dest):
        # Un-directed graph hai, toh dono taraf rasta banayenge
        self.matrix[source][dest] = 1
        self.matrix[dest][source] = 1

    def bfs(self, start_vertex):
        # Sabhi nodes ko pehle 'False' (unvisited) mark kiya
        visited = [False] * self.size

        # Queue banayi aur start node ko dala
        q = deque([start_vertex])
        visited[start_vertex] = True # Visited mark kiya

        print("BFS Traversal Order: ", end="")

        # Jab tak Queue empty na ho
        while q:
            # Front se ek element nikala (left side se)
            v = q.popleft()
            print(v, end=" ")

            # Matrix ki uss row (v) mein sabhi padosiyon ko check karo
            for i in range(self.size):
                # Agar edge hai (1) aur node abhi visited nahi hai
                if self.matrix[v][i] == 1 and not visited[i]:
                    visited[i] = True # Visited mark karo
                    q.append(i) # Queue mein last mein dalo

# Graph banate hain (Video wala same example)
g = Graph(8)
g.add_edge(0, 1)
g.add_edge(0, 3)
g.add_edge(1, 3)

```

```
g.add_edge(3, 4)
g.add_edge(3, 5)
g.add_edge(4, 5)
g.add_edge(4, 6)
g.add_edge(6, 2)
g.add_edge(6, 7)

# Traversal Start Karte Hain
g.bfs(0)
# Output aayega: 0 1 3 4 5 6 2 7
```



Important Points to Remember

- **DFS vs BFS:** DFS ek raste par tab tak andar (depth) tak jata hai jab tak dead-end na aa jaye. BFS apne aas-paas (breadth) sab kuch pehle explore karta hai.
- **Queue ka Magic:** BFS hamesha Queue (FIFO) data structure use karta hai kyunki humein sequence maintain karna hota hai.
- **Shortest Path:** Kisi bhi jagah ka sabse chhota rasta (shortest route) nikalne ke liye BFS algorithm best hai.
- **Multiple Answers:** Ek hi level ke nodes ka order aage-peeche ho sakta hai (jaise [1,3] ki jagah [3,1]), aur dono hi answers bilkul sahi mane jayenge.